In [1]:
```
# ------------------------------
# Object Detection using Transfer Learning (VGG16 + CIFAR-10)
# ------------------------------

import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.utils import to_categorical
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1793421975.017680   11205 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them o
ff, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1793421975.021666   11205 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
I0000 00:00:1793421975.474364   11205 cpu_feature_guard.cc:227] This Tenso
rFlow binary is optimized to use available CPU instructions in performanc
e-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operatio
ns, rebuild TensorFlow with the appropriate compiler flags.
/usr/lib/python3/dist-packages/scipy/__init__.py:146: UserWarning: A NumPy
version >=1.17.3 and <1.25.0 is required for this version of SciPy (detect
ed version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}"
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1793421976.681394   11205 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floatin
g-point round-off errors from different computation orders. To turn them o
ff, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1793421976.682857   11205 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
```

In [2]:
```
# Step 1: Load CIFAR-10 dataset
(itrain, ltrain), (itest, ltest) = cifar10.load_data()
```

In [3]:
```
# Normalize images (0-255 → 0-1)
itrain = itrain / 255.0
itest = itest / 255.0

# One-hot encode labels
ltrain = to_categorical(ltrain)
ltest = to_categorical(ltest)
```

In [4]:
```
# Step 2: Load pre-trained VGG16 model (without top layers)
basem = VGG16(weights='imagenet', include_top=False, input_shape=(32, 32,
```

```
E0000 00:00:1793421999.541861   11205 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

In [5]:
```
# Step 3: Freeze pre-trained layers
for layer in basem.layers:
    layer.trainable = False
```

In [ ]:
```
# Step 4: Create new model on top of VGG16
semodel = Sequential()
semodel.add(basem)
semodel.add(Flatten())
semodel.add(Dense(256, activation='relu'))
semodel.add(Dense(10, activation='softmax'))  # CIFAR-10 → 10 classes
```

In [ ]:
```
# Step 5: Compile the model
semodel.compile(optimizer='adam', loss='categorical_crossentropy', metric
```

In [ ]:
```
# Step 6: Train the model
history = semodel.fit(
    itrain, ltrain,
    epochs=10,
    batch_size=32,
    validation_data=(itest, ltest)
)
```

In [ ]:
```
# Step 7: Evaluate model
loss, acc = semodel.evaluate(itest, ltest)
print("\nTest accuracy:", acc)
```